

SENTINEL: Evasion-Aware Trust Scoring, Deception, and Cryptographic Write Authorization for Industrial IoT

Mohammad Salman Khan*, Hammad Chughtai†, Halil İbrahim Özkul*, and Burak Yılmaz*

*Department of Software Engineering, Konya Technical University, Konya, Turkey

†Department of Computer Science (Cyber Security), University of Bradford, Bradford, United Kingdom

Abstract—Industrial Internet of Things (IIoT) environments require security controls that can distinguish legitimate operational changes from credential misuse, gradual process manipulation, and unauthorized command execution without violating real-time and availability constraints. Detection alone is insufficient when a compromised network-facing service can still issue writes to operational technology. This paper presents SENTINEL, an architecture that separates a public command broker from a protected trust decision point, routes uncertain sessions to a session-isolated deception environment, and requires a short-lived, target-bound cryptographic capability before an actuator or process setpoint can be modified. The trust scorer evaluates a 48-command window using identity, timing, authorization, replay, and process-context features. We evaluate a reference implementation on a reproducible synthetic corpus of 192,000 commands organized into 4,000 complete sessions. The split is performed by session, and the held-out test set introduces later attack onset, benign-profile mixing, and maintenance-like benign transients to reduce same-generator optimism. The resulting scorer achieves 95.33% accuracy, 90.82% macro-F1, and a 0.28% benign false-positive rate. A two-threshold policy approves 99.17% of benign sessions and contains 98.33% of attack sessions, with no benign session immediately blocked in the test set. In 14,000 token-misuse trials, all missing, modified, expired, replayed, mis-scoped, unauthorized-role, and unsafe-value requests were rejected. On a five-vCPU AMD EPYC host, the complete in-process feature and scoring path has 5.10 ms median and 7.87 ms p99 latency. These results support the architectural feasibility of combining behavioral trust, deception, and enforced write authorization, while also showing that physical-plant validation remains necessary before deployment in safety-critical environments.

Index Terms—Industrial IoT, operational technology, trust scoring, cyber deception, honeypot, command authorization, replay prevention, insider threat, evasion attack.

I. INTRODUCTION

Operational technology (OT) increasingly depends on interconnected sensors, programmable logic controllers (PLCs), remote engineering workstations, gateways, and cloud or enterprise services. This convergence improves observability and automation but exposes physical processes to software and identity compromise. OT defenses must therefore protect not only information assets, but also actions that can change a physical process. NIST emphasizes that OT security controls must preserve safety, reliability, and performance requirements that differ from conventional enterprise systems [1].

A recurring weakness in industrial architectures is that a successfully authenticated session is treated as trustworthy for

the remainder of its interaction. Stolen credentials, a malicious insider, or a compromised network-facing service can then issue commands that are individually valid but collectively unsafe. Slow manipulation is especially difficult because an adversary can remain within static ranges and spread a process change across many commands. Prior work has shown that stealthy industrial attacks can trade impact against detectability and can evade monitors that consider only isolated observations [5].

Three families of defenses address parts of this problem. Network intrusion detection systems, such as Kitsune, model packet-level or channel-level behavior [6]. Multivariate time-series methods capture temporal and cross-sensor relationships [10]–[12]. Industrial honeypots, including Conpot, provide deception and intelligence collection [14], [15]. These approaches are valuable, but they usually operate in a detect-and-alert mode. They do not necessarily ensure that every physical write is authorized by an isolated policy decision point, and they generally do not use deception as an intermediate response to uncertainty.

This paper presents SENTINEL, a command-path architecture that combines four controls. First, a public broker cannot independently authorize a write. Second, a protected trust decision point evaluates policy and recent behavioral context. Third, uncertain sessions are transparently redirected to a session-isolated decoy state instead of being immediately blocked. Fourth, approved writes require a short-lived capability token that is bound to the exact worker, role, target, operation, value, nonce, and policy version. The design follows the zero-trust principle that access decisions should be made for the protected resource rather than inferred from network location [2].

The contributions are:

- A separation of the network-facing broker, trust decision point, deception environment, and OT enforcement point, with explicit trust boundaries and failure assumptions.
- A command-window trust scorer that combines identity, authorization, timing, replay, and process-context evidence and maps risk into *approve*, *deceive*, or *block* outcomes.
- A target-bound HMAC capability and one-time nonce protocol that prevents payload substitution, stale-token use, replay, and cross-device token reuse at an OT gateway.
- A reproducible session-level simulation study with a shifted

held-out test domain, comparative baselines, feature ablations, token misuse tests, and CPU latency measurements.

The paper deliberately separates three claims that are often conflated in IIoT security research. *Detection* concerns whether a command sequence is recognized as suspicious; *containment* concerns whether uncertain or malicious activity is prevented from reaching the physical process; and *authorization integrity* concerns whether a network-facing compromise can substitute, replay, or redirect an otherwise valid write. This separation allows a system to remain useful even when the classifier is uncertain: a session can be contained in deception without claiming perfect attack attribution.

The remainder of the paper is organized as follows. Section II reviews temporal detection, industrial deception, and resource-centric authorization. Section III defines the assets, design goals, adversary capabilities, and safety assumptions. Sections IV and V present the architecture, algorithms, implementation choices, and deployment patterns. Sections VI and VII describe the evaluation and results. Sections VIII and IX discuss security implications, threats to validity, and the work required for hardware-in-the-loop validation.

II. BACKGROUND AND RELATED WORK

A. Industrial detection and datasets

Public testbeds and datasets have enabled repeatable ICS security research. SWaT provides a six-stage water-treatment testbed and associated attack data [8], [9]; TON_IoT provides telemetry and network data for IoT and IIoT intrusion detection [7]. These resources are important, but their recorded variables do not directly represent all of the command-authorization fields required by SENTINEL, such as worker role, nonce freshness, request target binding, and session-specific deception state. The present study therefore uses a transparent synthetic command generator and treats real-plant validation as future work rather than presenting synthetic results as field performance.

Kitsune demonstrates efficient online network anomaly detection through an ensemble of autoencoders [6]. USAD, GDN, and TranAD model multivariate temporal behavior using adversarial autoencoders, learned graphs, and transformers, respectively [10]–[12]. These models focus primarily on anomaly detection. In contrast, SENTINEL’s primary unit of control is an authenticated command request, and the output is an enforcement decision rather than an alarm alone.

B. Deception and honeypots

Conpot emulates industrial protocols to collect intelligence on attackers targeting ICS services [14]. Surveys show that industrial honeypots can support threat intelligence and defensive experimentation, but also face realism and fingerprinting limitations [15]. Recent work demonstrates that observable network characteristics can reveal ICS honeypots, reinforcing the need to describe deception fidelity conservatively rather than claim indistinguishability [16].

Shadow honeypots combine anomaly detection with isolated execution and discard state changes caused by malicious requests [13]. SENTINEL adopts a related principle at the

command-session level: uncertain commands are executed only against a decoy process state, while the live OT state remains unchanged. Unlike a general software shadow, the decoy is integrated with command authorization and an OT write enforcement point.

C. Resource-centric authorization and zero trust

Zero-trust architecture does not imply that every request must traverse a cloud identity platform; rather, it requires that access to a protected resource be decided from authenticated identity, device, policy, and context instead of network location alone [2]. In an IIoT command path, the protected resource is not merely an API endpoint. It is the ability to mutate an actuator, PLC register, configuration parameter, or process setpoint. Consequently, an application-layer “allow” decision is insufficient unless a policy enforcement point adjacent to the OT resource verifies the decision.

SENTINEL implements this separation through a narrow write capability. The capability resembles a message authentication code over an immutable command context and is accepted only once during a short validity interval. HMAC provides integrity and source authentication when the decision point and enforcement point can share a protected key [3]. This mechanism is complementary to anomaly detection: hard authorization rejects commands that are invalid independent of behavior, while the scorer handles context-dependent cases that are individually legal but collectively suspicious.

D. Gap addressed by this work

The literature contains strong components for network anomaly detection, multivariate process monitoring, deception, and policy enforcement, but these components are usually evaluated independently. A detector may report a suspicious session without controlling whether its next write reaches the plant; a honeypot may attract scanning traffic without being part of a live command decision; and a cryptographic gateway may verify integrity without detecting a legitimate credential being used gradually and maliciously. The research question addressed here is therefore architectural as well as predictive: can these controls be composed into one command lifecycle with measurable trade-offs among availability, uncertainty handling, and write prevention?

E. Positioning

Table I compares the principal capabilities relevant to this work. The table avoids subjective readiness scores; it records whether each work directly supports the listed function.

III. SYSTEM AND THREAT MODEL

A. Design objectives

The architecture is guided by six objectives. **G1: mandatory mediation** requires every remote write to pass through a verifier adjacent to the OT resource. **G2: broker compromise tolerance** requires Model B to remain unable to create a valid physical-write authorization by itself. **G3: temporal context** requires the decision point to evaluate a sequence rather than

TABLE I
CAPABILITY POSITIONING OF REPRESENTATIVE APPROACHES

Approach	Command-level context	Temporal behavior	Enforced write gate	Active deception	OT/IIoT focus	Primary role
Kitsune [6]	×	✓	×	×	△	Network anomaly detection
USAD / GDN / TranAD [10]–[12]	△	✓	×	×	△	Multivariate anomaly detection
Conpot [14]	×	×	×	✓	✓	ICS honeypot and intelligence collection
Shadow honeypots [13]	△	△	×	✓	×	Isolated validation of suspicious software requests
SENTINEL	✓	✓	✓	✓	✓	Command decision, deception, and write enforcement

✓ = directly supported; △ = partial or indirect support; × = not a native function.

a single command. **G4: uncertainty containment** requires suspicious but inconclusive sessions to be isolated without immediately disrupting all legitimate activity. **G5: bounded overhead** requires the decision path to remain compatible with supervisory command rates. **G6: auditable evidence** requires decisions, token outcomes, and decoy interactions to be reconstructable without automatically poisoning the trusted baseline.

B. Protected assets and security invariants

The protected assets are the live actuator/setpoint state, worker authorization context, session decision state, forensic evidence, and signing material. The central invariant is that live OT state changes only after the enforcement point verifies a fresh capability whose authenticated fields exactly match the pending command. Identity is signed independently of Model B; uncertainty is isolated in a session-specific decoy namespace; audit evidence is append-only; and the broker never possesses the capability-signing key.

C. System entities

The architecture contains five security-relevant entities:

- 1) **Ingress identity service.** Authenticates the worker and device, validates request freshness, and produces a signed identity context.
- 2) **Model B, public command broker.** Parses protocol messages, normalizes commands, and maintains session transport state. It does not make final trust decisions.
- 3) **Model A, trust decision point.** Applies hard policy constraints, computes a behavioral score, chooses an action, and signs write capabilities.
- 4) **Deception service.** Maintains a session-isolated decoy state and returns protocol-consistent acknowledgments without changing the live process.
- 5) **OT enforcement point.** Validates the capability, nonce, target, role, value, and expiration before forwarding a write to a PLC, actuator, or legacy gateway.

Fig. 1 shows the command flow, while Fig. 2 separates the public, decision, and OT enforcement zones.

D. Adversary capabilities

We consider five operational attack classes and one architectural compromise. An insider may issue legal commands whose cumulative effect is unsafe; stolen credentials may be used from an unusual origin or cadence; a slow-evasion adversary may spread low-amplitude changes over time; an authenticated worker may exceed role permissions; and a replay attacker may repeat, retarget, or modify captured commands. We also allow arbitrary code execution in Model B, including payload alteration and reordering. The broker cannot forge the independently signed ingress identity context or the capability token because it holds neither signing key. The evaluation is closest to a grey-box attacker: the command flow and feature categories are known, but protected secrets and the exact trained model are not directly available.

E. Trusted computing base and exclusions

The trusted computing base contains Model A, the capability-signing key, the OT enforcement point, and the identity-signing service. Physical tampering, compromise of Model A, compromise of the enforcement gateway, and denial-of-service against the control channel are outside the experimentally evaluated threat model. The behavioral scorer cannot guarantee detection of a perfectly in-distribution attack that produces no measurable identity, temporal, authorization, or process-context deviation. These assumptions follow the adversarial-ML distinction between attacker knowledge, capability, and lifecycle access [4].

F. Safety and failure behavior

A security gateway must not replace local safety interlocks. If Model A is unavailable, remote writes fail closed, while local PLC safety logic, emergency shutdown, and manually authorized maintenance procedures remain independent. Reads may continue through a degraded path when organizational policy permits. A maintenance bypass, when required, must be explicit, time-bounded, physically or independently approved,

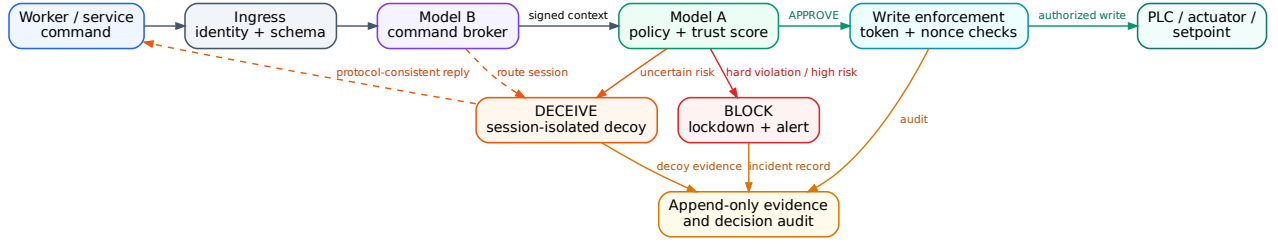


Fig. 1. SENTINEL command path. Model B is a broker, not an authorization authority. Model A returns one of three decisions: approve with a scoped capability, deceive through a session-isolated decoy state, or deny and alert.

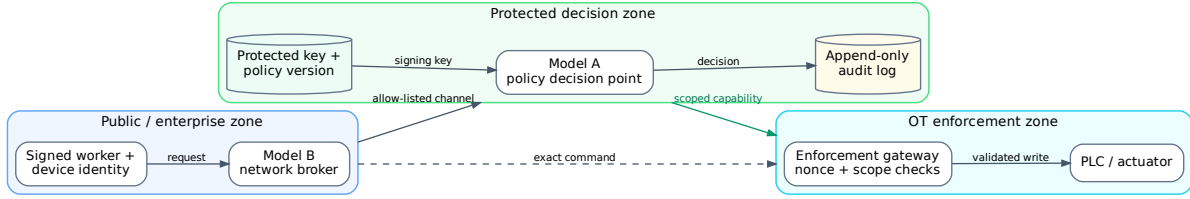


Fig. 2. Trust boundaries and protected assets. The signing key is confined to the decision zone. A legacy PLC can be protected by an enforcement gateway even when the PLC cannot validate tokens natively.

and logged; it must not be implemented as a silent fail-open condition.

The reference prototype evaluates command integrity and authorization; it does not claim to certify process safety. A production deployment must define the safe state for each actuator, the maximum tolerated decision delay, clock-skew bounds, gateway restart behavior, and what happens to unused capabilities when a policy or key version changes.

IV. SENTINEL DESIGN

A. End-to-end command lifecycle

A command passes through six logical steps. First, the ingress service authenticates the worker and produces a signed identity context containing the worker, role, client/device identifier, issue time, and session identifier. Second, Model B parses the public protocol and converts protocol-specific fields into a canonical command. Third, Model A verifies the identity context and applies hard role, target, freshness, and safety constraints. Fourth, commands that pass hard checks are scored against the active session window. Fifth, the decision is mapped to live approval, deception, or denial. Finally, the OT enforcement point independently validates the capability before forwarding the write. No response from Model B alone is sufficient to change the live state.

B. Command context and hard constraints

A normalized command at time t is represented as

$$c_t = (u, r, d, o, v, n, t_i, t_e, p), \quad (1)$$

where u is the worker identity, r the role, d the target device or register, o the operation, v the value, n a nonce, t_i and t_e the issue and expiry times, and p the policy version. Before behavioral scoring, Model A rejects malformed requests, stale identity contexts, role violations, unsafe values, and operations outside the target policy. Hard violations are never routed to the live process.

The policy evaluation order is intentional. Syntax and freshness checks occur before any expensive model operation. Role and target authorization then establish whether the operation is categorically permitted. A device-specific safe envelope rejects values that are unacceptable regardless of behavioral history. Only a command that is individually admissible proceeds to contextual scoring. This arrangement avoids using a probabilistic model to override deterministic safety or authorization rules.

C. Windowed trust scorer

For requests that pass hard constraints, Model A maintains a 48-command session window. Each command contributes 19 features covering operation type, normalized value and delta, timing, time of day, role permission, origin trust, nonce freshness, repeated-command similarity, cross-sensor residual, safe-envelope status, rate deviation, session progress, and device type.

The window is mapped to a fixed vector using eight statistics per feature: mean, standard deviation, minimum, maximum, end-to-start difference, linear slope, first quartile, and third quartile. The resulting vector has $19 \times 8 = 152$

dimensions. Table II groups the command-level evidence by security purpose. The design intentionally combines features that are difficult for one compromised component to manipulate simultaneously: role permission is derived from policy, nonce freshness from the verifier, origin trust from ingress, and process residuals from relationships among device observations.

The deployed reference scorer is a 160-tree random forest with maximum depth 15, minimum leaf size 3, square-root feature sampling, and balanced subsampling. A tree ensemble was selected over a recurrent model because it provided stronger shifted-test performance and interpretable feature importance in the reference study. This choice also keeps inference deterministic for a fixed model and avoids claiming neural behavior that was not evaluated.

Let $P(y = k | \phi(S_t))$ be the class probability for the current session summary $\phi(S_t)$. The maliciousness score is

$$s_t = 1 - P(y = \text{normal} | \phi(S_t)). \quad (2)$$

The response is

$$D(s_t) = \begin{cases} \text{approve,} & s_t < \tau_{\text{clear}}, \\ \text{deceive,} & \tau_{\text{clear}} \leq s_t < \tau_{\text{block}}, \\ \text{block,} & s_t \geq \tau_{\text{block}}. \end{cases} \quad (3)$$

Thresholds are selected on the validation set under a constraint that immediate false blocks remain below 2%. Candidate pairs are ranked using

$$U = 1.6C_a + 0.8A_b + 0.25B_a - 3.2B_b, \quad (4)$$

where C_a is attack containment, A_b is benign approval, B_a is immediate attack blocking, and B_b is benign immediate blocking. The coefficients express the reference deployment preference for containment while strongly penalizing disruptive false blocks; they are not universal constants. The selected values are $\tau_{\text{clear}} = 0.45$ and $\tau_{\text{block}} = 0.65$.

D. Deception state and session transitions

The deception service allocates a state namespace per suspicious session. On the first diversion, the namespace is initialized from a sanitized snapshot of the values visible to the session. Subsequent reads and writes are resolved against that namespace, preserving request syntax, status codes, and read-after-write consistency. A decoy write never updates the live process, and one suspicious worker cannot observe another worker's decoy state.

The session remains in deception until one of four events occurs: the score crosses τ_{block} , the session expires, an operator explicitly terminates it, or a de-escalation policy confirms sufficient benign evidence. The reference experiment evaluates diversion and block decisions at the session level; it does not claim a complete industrial process simulator. The decoy is therefore not described as indistinguishable from a real plant. Its purpose is controlled observation and containment. Raw decoy interactions are stored as forensic evidence and are not automatically added to trusted training data, avoiding a direct poisoning path.

Algorithm 1 SENTINEL command decision

Require: command c_t , signed identity context I_t , session history S_t

```

1: if  $I_t$  is invalid or stale then
2:   return block and alert
3: end if
4: if  $c_t$  violates role, target, or safety policy then
5:   return block and alert
6: end if
7: append derived features of  $c_t$  to  $S_t$ 
8:  $s_t \leftarrow 1 - P(y = \text{normal} | \phi(S_t))$ 
9: if  $s_t < \tau_{\text{clear}}$  then
10:  issue target-bound capability  $T$ 
11:  return approve with  $T$ 
12: else if  $s_t < \tau_{\text{block}}$  then
13:  return route to session-isolated deception state
14: else
15:  return block, persist evidence, and alert
16: end if
```

E. Cryptographic write capability

For an approved command, Model A issues

$$T = \text{HMAC}_K(u \parallel r \parallel d \parallel o \parallel v \parallel n \parallel t_i \parallel t_e \parallel p). \quad (5)$$

The reference lifetime is 800 ms. The enforcement point recomputes the HMAC, verifies the exact target and value, checks role and safe-envelope constraints, validates the policy version and time interval, and rejects any previously used nonce. A token for one device, value, or operation cannot authorize another. HMAC is used in the reference prototype for low overhead; production deployments may use asymmetric capabilities or hardware-protected keys when organizational key distribution requires them.

F. Security properties

Under the stated trust assumptions, the command path provides four bounded properties. **P1: exact-command integrity.** Modification of any bound field causes the enforcement point's recomputed HMAC to differ. **P2: single-use freshness.** A captured capability cannot be accepted after expiry or after its nonce has been atomically consumed. **P3: broker non-authority.** Model B cannot independently authorize a write because it lacks the signing key and cannot produce an independently signed ingress identity context. **P4: uncertain-write isolation.** A deceive decision resolves the command against a session-specific namespace and therefore does not mutate live OT state.

These are implementation-level properties, not a proof of whole-system safety. They depend on key confidentiality, correct canonical serialization, atomic nonce handling, enforcement-point integrity, and the assumption that all remote writes are mediated. The design does not prevent a local physical override or a compromised PLC from altering its own state.

TABLE II
COMMAND-LEVEL FEATURES AGGREGATED OVER THE 48-COMMAND SESSION WINDOW

Group	Features	Security interpretation
Operation and target	read/write/setpoint indicators; pump/valve/heater indicators	Distinguishes ordinary monitoring from state-changing operations and captures device-specific patterns
Process context	normalized value, normalized delta, cross-sensor residual, safe-envelope flag	Detects cumulative drift and inconsistency between a requested value and related process observations
Identity and authorization	role-permission flag, origin-trust score	Identifies commands that conflict with the role or appear from an unusual authenticated context
Freshness and replay	nonce-freshness flag, repeated-command similarity	Captures stale or duplicated command material and replay-like sequences
Timing and workload	log inter-arrival time, rate deviation, time-of-day sine/cosine	Models cadence, bursts, shift changes, and unusual access times without treating one time value as categorically malicious
Session position	normalized session progress	Allows the scorer to distinguish benign startup/maintenance transients from evidence that emerges late in a session

Algorithm 2 OT enforcement-point verification

Require: pending command c , capability T , current time t , used-nonce set N

- 1: **if** T is absent or its key identifier is unknown **then**
- 2: **return** reject
- 3: **end if**
- 4: **if** $t < t_i - \delta$ or $t > t_e$ **then**
- 5: **return** reject
- 6: **end if**
- 7: **if** $n \in N$ or policy version p is inactive **then**
- 8: **return** reject
- 9: **end if**
- 10: **if** role, target, operation, or value violates local policy **then**
- 11: **return** reject
- 12: **end if**
- 13: $T' \leftarrow \text{HMAC}_K(u \parallel r \parallel d \parallel o \parallel v \parallel n \parallel t_i \parallel t_e \parallel p)$
- 14: **if** $T' \neq T$ using constant-time comparison **then**
- 15: **return** reject
- 16: **end if**
- 17: atomically add n to N
- 18: forward the exact pending write to the live OT endpoint
- 19: **return** accept and persist the verification record

G. Computational and storage complexity

Let W be the session-window length, F the number of command features, and M the number of trees. Updating the fixed-window statistics is $O(F)$ with incremental aggregates or $O(WF)$ when recomputed as in the reference script. Tree inference is $O(MD)$ for average tree depth D . The reference values $W = 48$, $F = 19$, $M = 160$, and $D \leq 15$ keep the path small enough for supervisory traffic. Per-session state consists of the recent command window, the current decision mode, decoy values when applicable, and a bounded set of unexpired nonces.

V. REFERENCE IMPLEMENTATION AND DEPLOYMENT

A. Software components

The reproducibility package implements the synthetic generator, feature aggregation, model training, threshold selection, capability tests, and latency benchmark in Python. The implementation stores each complete session as a 48×19 tensor and exports the trained scaler, class map, and random-forest model in one versioned artifact. The token verifier uses canonical field ordering and constant-time MAC comparison. Random seeds, split rules, model hyperparameters, and generated result files are included so that the reported synthetic experiment can be reproduced.

A production implementation can separate these functions by language and host without changing the protocol. Model B may be an HTTP, MQTT, OPC UA, or Modbus-facing adapter; Model A may run as an isolated policy service; and the enforcement point may be embedded in a protocol gateway. The critical requirement is not a specific framework but preservation of the trust boundaries and exact command binding.

B. Deployment patterns

Two deployment patterns are considered. In a *native-verifier* deployment, a capable PLC, secure edge controller, or actuator firmware validates the capability directly. This minimizes the distance between the protected state and the enforcement logic but requires firmware support and secure key storage. In a *legacy-gateway* deployment, a dedicated gateway is the only routable writer to a legacy PLC. The gateway validates the capability and then emits the protocol-specific command. This is easier to integrate but expands the trusted computing base to include the gateway and the network segment between gateway and PLC.

C. Key, nonce, and policy lifecycle

Capabilities include a key identifier and policy version so that rotation can occur without accepting commands under an ambiguous policy. During key rotation, the enforcement point

may temporarily accept the current and immediately previous key identifiers, subject to their original short expirations. Nonces must be consumed atomically before forwarding the live write; otherwise concurrent duplicate submissions could race. A restart-safe implementation persists or reconstructs all still-valid nonces, or invalidates all outstanding capabilities by advancing the key or policy version.

Audit records should contain the canonical command hash, identity-context identifier, model version, score, selected action, capability identifier, verifier result, and monotonic event time. Sensitive worker identifiers can be pseudonymized in analytical exports while remaining recoverable under controlled incident-response procedures.

VI. EXPERIMENTAL METHODOLOGY

A. Research questions

The evaluation addresses four questions:

- RQ1: Can the scorer distinguish normal sessions from five command-level attack classes under a shifted test domain?
- RQ2: What is the operational trade-off among approval, deception, and immediate block thresholds?
- RQ3: Does the capability verifier reject common token and payload misuse cases?
- RQ4: Is the in-process decision path compatible with low-millisecond command handling?

B. Synthetic command corpus

The corpus contains 4,000 complete sessions and 192,000 commands. Sixty percent of sessions are normal; each of five attack classes contributes 8%. Attack sessions begin after a benign prefix and then introduce one of the following patterns:

- **Insider drift:** gradual setpoint change with increasing cross-sensor residual.
- **Credential misuse:** origin and cadence mismatch with mostly valid values.
- **Slow evasion:** low-amplitude cumulative drift that remains within a static safe envelope.
- **Privilege abuse:** intermittent writes outside the worker role.
- **Replay:** repeated commands with stale nonces and elevated similarity.

Normal sessions include remote-maintenance, process-transient, and repeated-polling episodes so that single anomalous features are not sufficient for classification. Each session has a randomized role, device type, starting value, cadence, start hour, origin-trust baseline, and read/write/setpoint distribution. Attack onset is sampled between commands 20 and 33 in the base generator, ensuring a benign prefix. Table III summarizes the evidence injected by each attack template. Values are clipped to a normalized range and static unsafe-envelope violations are deliberately rare in the drift classes; this prevents a simple range rule from solving the task.

The split is stratified by complete session: 70% training, 15% validation, and 15% testing. No command window from one session appears in more than one split. The test set is further shifted by delaying attack evidence, mixing attack traces with independent benign profiles, and injecting mild

maintenance-like features into some normal sessions. This shift is intentionally modest: it tests whether the classifier survives overlap and later evidence, but it is not equivalent to an independently collected plant. Table IV summarizes the split.

C. Baselines and hyperparameters

We compare the SENTINEL scorer against fixed rules, balanced multinomial logistic regression, and histogram gradient boosting. All learned models use the same 152-dimensional session summaries and the same session splits. Table V records the configuration used in the reproducibility package. The rules baseline uses hand-written thresholds on role violations, origin trust, nonce freshness, repeated-command similarity, value drift, process residual, and rate deviation. It represents a plausible low-complexity alternative rather than a published system whose input representation differs from the command corpus.

D. Metrics and statistical analysis

Metrics include accuracy, balanced accuracy, macro precision, macro recall, macro-F1, benign false-positive rate (FPR), binary ROC-AUC, and binary PR-AUC. Per-class precision, recall, and F1 are reported because aggregate accuracy is dominated by the normal class. Binary AUC treats all five attack classes as one positive class and therefore measures separation of benign from malicious sessions, not attack attribution.

To express test-set sampling uncertainty, we perform 2,000 nonparametric bootstrap resamples of the 600 held-out sessions and report percentile 95% confidence intervals for accuracy, macro-F1, FPR, and attack containment. These intervals do not capture generator uncertainty, model-development choices, or deployment shift; they quantify only resampling uncertainty within the fixed test corpus.

E. Capability and latency tests

The verifier is tested with 2,000 attempts for each of seven misuse cases: missing token, modified value, expired token, wrong target, replayed nonce, unauthorized role, and unsafe value, for 14,000 unauthorized attempts total. A valid control case is executed first. Each negative case changes one relevant condition while preserving the others, allowing the rejection reason to be attributed to the corresponding verification step.

Latency is measured on an AMD EPYC 9V74 virtual host with five vCPUs and 5.9 GiB RAM using Python 3.13 and scikit-learn 1.8. The scorer benchmark uses one model thread after warm-up; the complete feature-extraction, scaling, and scoring pipeline uses 1,000 invocations. HMAC issue and verification are measured separately. Measurements exclude network transport, queueing, database access, audit persistence, protocol translation, and PLC response time. The reported values are therefore computational microbenchmarks rather than end-to-end plant latency.

TABLE III
SYNTHETIC ATTACK TEMPLATES AND DISCRIMINATING EVIDENCE

Class	Generated behavior after onset	Primary evidence available to scorer	Why a static rule is insufficient
Insider drift	Incremental setpoint movement and gradually increasing process residual	Value slope, end-to-start change, residual trend and dispersion	Most individual values remain within the safe envelope and originate from a permitted worker
Credential misuse	Lower/more variable origin trust, cadence change, modest workload-rate increase	Origin-trust minimum/trend, inter-arrival statistics, rate deviation	Command syntax, role, and values are usually valid
Slow evasion	Smaller cumulative drift with oscillation and weak residual growth	Long-window value and residual slopes	Each update is intentionally below a conspicuous per-command threshold
Privilege abuse	Intermittent write/setpoint operations for which role permission is false	Role-permission minimum and variation, operation mix	Violations are sparse and can be surrounded by ordinary reads
Replay	Repeated operation/value templates, stale nonces, elevated similarity	Nonce freshness and repeated-command statistics	Replayed commands may contain otherwise safe and historically common values

TABLE IV
SYNTHETIC SESSION CORPUS

Class	Total	Train	Val.	Test
Normal	2400	1680	360	360
Insider drift	320	224	48	48
Credential misuse	320	224	48	48
Slow evasion	320	224	48	48
Privilege abuse	320	224	48	48
Replay	320	224	48	48
Total sessions	4000	2800	600	600

TABLE V
EVALUATED MODEL CONFIGURATIONS

Model	Configuration
Rules	Seven feature-derived threshold conditions; first matching attack label returned
Logistic regression	Multinomial, balanced class weights, $C = 1.2$, maximum 700 iterations
Histogram boosting	180 iterations, learning rate 0.07, 25 maximum leaf nodes, $L_2 = 0.15$, balanced weights
SENTINEL random forest	160 trees, depth 15, minimum leaf 3, square-root feature sampling, balanced subsampling

F. Reproducibility protocol

The artifact records the random seed, generated class counts, complete-session split, trained model, feature names, threshold values, per-class metrics, ablations, token outcomes, latency samples, and figure-generation scripts. Re-running the package regenerates the synthetic corpus rather than relying on manually entered tables. The package does not contain field data and should not be presented as evidence of a deployed industrial installation.

VII. RESULTS

A. Classification performance

Table VI reports the shifted-test results. The SENTINEL scorer achieves 95.33% accuracy, 90.82% macro-F1, and a 0.28% FPR. Gradient boosting is close, while the linear and rule baselines fail to separate several temporally overlapping classes. The high binary AUC values should be interpreted

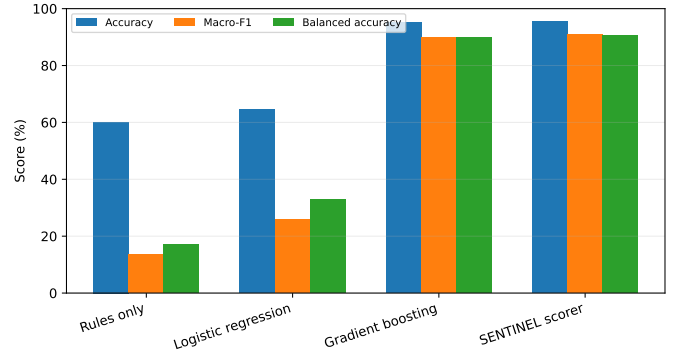


Fig. 3. Accuracy, macro-F1, and balanced accuracy under the shifted test domain.

alongside the lower multiclass macro-F1: distinguishing attack from normal is easier than identifying the attack type.

Bootstrap intervals in Table VII show that the headline result remains high under resampling of the fixed test sessions, although the interval for macro-F1 is wider because each attack class contains only 48 test sessions.

Fig. 4 and Fig. 5 show class-level behavior. Replay, credential misuse, privilege abuse, and slow evasion have recall of 100.0%, 97.9%, 95.8%, and 95.8%, respectively. Insider drift is the weakest class at 54.2% recall because its late, low-amplitude evidence overlaps with benign process transients. This result is more informative than the aggregate accuracy and identifies a clear target for future process-model integration.

The score distribution shows strong benign/attack separation, with a small overlap near the clear threshold for credential-misuse, slow-evasion, and privilege-abuse sessions. This overlap motivates a deception band rather than an all-or-nothing classifier threshold.

B. Triage behavior

At $\tau_{\text{clear}} = 0.45$ and $\tau_{\text{block}} = 0.65$, 99.17% of benign sessions are approved, 0.83% enter deception, and none are immediately blocked. Among attack sessions, 98.33% are contained by either deception or block, 96.25% are immediately blocked, and 1.67% are missed. Fig. 6 shows that the two-

TABLE VI
SHIFTED-TEST CLASSIFICATION RESULTS (%)

Model	Acc.	Bal. Acc.	Macro-P	Macro-R	Macro-F1	FPR	ROC-AUC	PR-AUC
Rules only	60.17	17.01	27.21	17.01	13.60	0.00	—	—
Logistic regression	64.67	33.01	54.59	33.01	25.79	6.11	92.66	90.15
Gradient boosting	95.00	89.88	93.53	89.88	89.87	0.28	99.95	99.92
SENTINEL scorer	95.33	90.58	94.22	90.58	90.82	0.28	99.87	99.83

TABLE VII
SESSION-BOOTSTRAP 95% CONFIDENCE INTERVALS

Metric	Point	Lower	Upper
Accuracy (%)	95.33	93.50	97.00
Macro-F1 (%)	90.82	87.58	93.66
Benign FPR (%)	0.28	0.00	0.86
Attack containment (%)	98.33	96.57	99.61

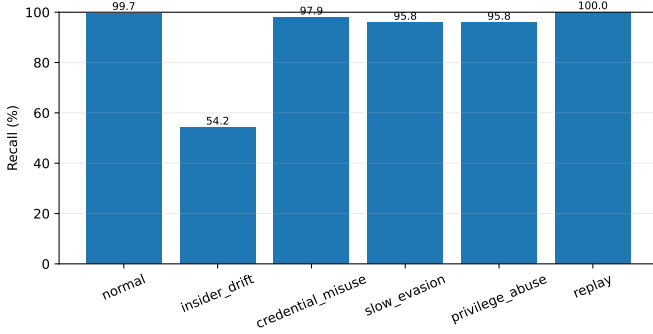


Fig. 4. Per-class recall for the SENTINEL scorer.

threshold policy exposes an operational trade-off that a single threshold would hide.

Slow evasion produces the largest deception proportion, which is consistent with the intended role of the middle band: sessions with gradual evidence receive additional isolation and observation rather than immediate live access. Privilege-abuse misses occur because the synthetic role violation is intermittent; in a production design, the hard policy stage should reject each disallowed operation even when the session-level classifier remains uncertain.

C. Feature ablation

Table IX evaluates the contribution of feature groups. Removing identity and command-context features reduces macro-F1 from 90.62% to 60.78%; removing process-context features reduces it to 69.90%. Means alone retain 86.40% macro-F1, but static extrema alone perform poorly and raise the FPR to 10.83%. The result supports the central design claim that authorization and process evidence should be evaluated together over time.

The dominant features are the slope and dispersion of cross-sensor residual, the trend and minimum of origin trust, role-permission variation, repeated-command similarity, and nonce

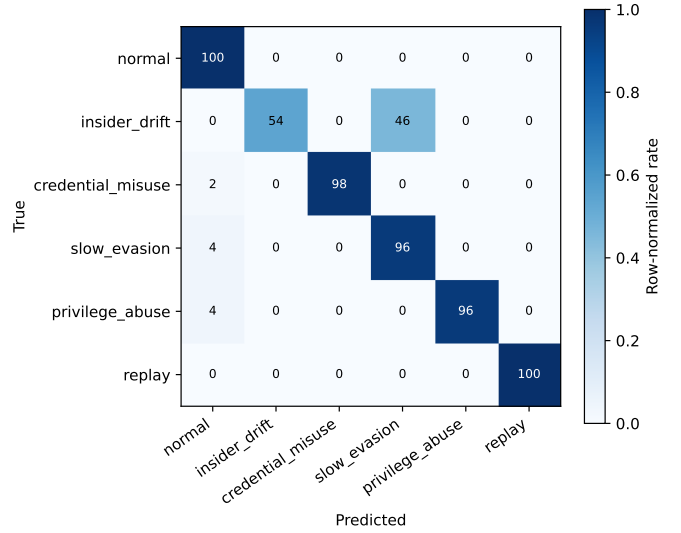


Fig. 5. Row-normalized confusion matrix. Values are percentages.

TABLE VIII
DECISION DISTRIBUTION BY TRUE CLASS (%)

Class	Approve	Deceive	Block
Normal	99.17	0.83	0.00
Insider drift	0.00	0.00	100.00
Credential misuse	2.08	2.08	95.83
Slow evasion	2.08	8.33	89.58
Privilege abuse	4.17	0.00	95.83
Replay	0.00	0.00	100.00

freshness. This distribution is consistent with the intended multi-source trust design rather than dependence on a single static threshold.

D. Error analysis

The principal attribution error is confusion between insider drift and slow evasion. Both classes modify values gradually and increase the cross-sensor residual; their difference is primarily attack amplitude and onset profile. For containment, this confusion is benign because both labels map to a non-normal score. For forensic attribution, however, the distinction is underdetermined from command summaries alone. A process-aware residual model, plant-mode indicator, or graph relating dependent sensors would provide stronger evidence.

The small number of normal sessions near τ_{clear} contain maintenance-like transients deliberately mixed into the test set.

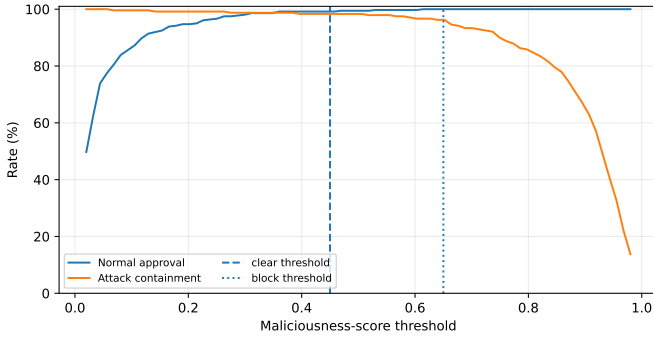


Fig. 6. Approval and containment rates as the maliciousness threshold changes.

TABLE IX
FEATURE ABLATION ON THE SHIFTED TEST SET

Configuration	Features	Macro-F1 (%)	FPR (%)
Full model	152	90.62	0.28
No identity/context	120	60.78	4.44
No process context	120	69.90	4.72
Mean only	19	86.40	3.61
Static extrema only	38	67.05	10.83

This is operationally important: a low FPR measured only on stationary normal traffic can substantially underestimate disruption during calibration, shift changes, or remote maintenance. Long-duration benign evaluation should therefore report false diversions and blocks per hour and stratify them by plant mode.

E. Capability enforcement

All 14,000 unauthorized attempts were rejected. The verifier accepted a valid command and rejected every missing-token, modified-value, expired-token, wrong-device, replay, unauthorized-role, and unsafe-value attempt. These tests establish implementation correctness for the evaluated misuse cases; they do not prove resistance to key extraction or compromise of the trusted enforcement point.

F. Latency

Table XI reports in-process CPU latency. The complete feature, scaling, and scoring path has a median of 5.10 ms and p99 of 7.87 ms. HMAC issue and verification are negligible relative to tree inference. These values do not include transport or device time and should not be interpreted as end-to-end plant latency.

VIII. SECURITY DISCUSSION

A. Compromised Model B

Model B compromise alone does not authorize a live write because the enforcement point requires both an exact command payload and a capability signed by Model A. To prevent Model B from inventing a worker identity, the identity context must be signed independently at ingress and verified by Model A. The security claim therefore depends on key separation and enforcement placement, not simply on network isolation.

TABLE X
CAPABILITY MISUSE TESTS

Case	Attempts	Accepted
Missing token	2000	0
Modified value	2000	0
Expired token	2000	0
Wrong device	2000	0
Replay	2000	0
Unauthorized role	2000	0
Unsafe value	2000	0

TABLE XI
IN-PROCESS LATENCY ON FIVE AMD EPYC VCPUS (MS)

Stage	Mean	P50	P95	P99
Scorer only	4.797	4.678	5.281	6.667
Full feature + scorer path	5.277	5.100	6.051	7.871
HMAC issue + verify	0.0030	0.0028	0.0040	0.0048

B. Replay and substitution

Binding the token to the command value, target, operation, worker, role, nonce, expiry, and policy version prevents payload substitution and cross-target reuse. A nonce cache prevents a valid token from being used twice during its 800 ms lifetime. Deployments must define bounded clock skew, key rotation, cache persistence after gateway restart, and safe behavior during policy-version changes.

C. Deception risks

An attacker may fingerprint a decoy through timing, process-physics inconsistencies, cross-session observations, or protocol details. SENTINEL therefore treats deception as containment and evidence collection, not as a guarantee that the adversary cannot identify the decoy. Higher-fidelity process simulation and network-stack equalization are necessary for advanced adversaries [16].

D. Adversarial learning and poisoning

The reference scorer is supervised and can be affected by distribution shift, label error, or poisoning. Evidence collected from a suspicious session is never promoted automatically into the trusted baseline. Model updates should use versioned, reviewed data; performance should be monitored by worker, device, and plant mode; and rollback should be possible. The present work evaluates a shifted test set but does not provide certified robustness.

E. Availability, keys, and protocol integration

Mandatory mediation creates an availability dependency on Model A and the enforcement point. Flooding, session-state exhaustion, and repeated scoring should be limited through early validation, per-identity quotas, bounded state, priority classes, and replicated decision services. Remote writes fail closed, while local safety and manual-control paths remain independent.

HMAC protects integrity only while the shared secret remains confidential. Production deployments should use protected key storage, key identifiers, rotation, and restart-safe nonce handling; asymmetric signatures may be substituted when shared-key distribution is unsuitable. Protocol adapters must also produce an unambiguous canonical command. For example, a Modbus binding should cover unit identifier, function, address, and value, while an OPC UA binding should cover endpoint, node, operation, and typed value. Audit records should preserve the command hash, identity context, model and policy versions, score, action, and verifier outcome.

IX. THREATS TO VALIDITY AND LIMITATIONS

The synthetic labels represent designed attack mechanisms, not ground-truth malicious intent in a real organization. Origin trust, process residual, role permission, and operating modes are simplified numeric signals. The generator, feature design, and model were developed within the same study, creating generator-model coupling. Complete-session splitting, benign transients, and a hardened test transformation reduce leakage, but they do not provide the independence of a separately collected plant dataset. Bootstrap intervals quantify only resampling uncertainty in the fixed 600-session test set.

The evaluation is command-centric and does not reproduce PLC scan cycles, network partitions, electromagnetic interference, operator workload, or protocol-specific behavior. The latency benchmark excludes network, queue, audit persistence, and device response time, and the verifier is modeled at a gateway rather than executed in PLC firmware. Adaptive, coordinated, and perfectly in-distribution long-horizon attacks remain outside the demonstrated coverage.

X. FUTURE WORK AND VALIDATION ROADMAP

The next stage is hardware-in-the-loop validation using a physical PLC or high-fidelity testbed with real protocol timing and process constraints. A complete study should measure false diversions and blocks per hour over long benign runs; end-to-end latency through protocol ingress, audit persistence, gateway verification, and PLC acknowledgement; concurrent replay races, clock skew, key rotation, restart behavior, and Model A failover. Model evaluation should add multiple seeds, unseen workers and devices, independently designed attacks, and process-aware graph residuals for insider drift. Deception should be tested for timing fidelity, cross-sensor consistency, multi-session isolation, and fingerprint resistance.

XI. CONCLUSION

SENTINEL combines mandatory write mediation, deterministic policy checks, temporal trust scoring, uncertainty-aware deception, and target-bound cryptographic authorization. In the reproducible synthetic study, the scorer achieved 95.33% accuracy, 90.82% macro-F1, and a 0.28% benign FPR. The selected triage policy contained 98.33% of attack sessions while approving 99.17% of benign sessions, and all 14,000 evaluated token-misuse attempts were rejected. Insider drift

remains difficult to attribute separately from slow evasion, although its maliciousness score is sufficient for containment.

The results support architectural feasibility under the stated simulation assumptions, not field readiness or safety certification. Real protocol traffic, independent attack generation, long-duration benign operation, and hardware-in-the-loop testing remain necessary. The artifact package includes the generator, trained scorer, metrics, token and latency tests, and figure scripts.

REFERENCES

- [1] K. Stouffer et al., "Guide to Operational Technology (OT) Security," NIST Special Publication 800-82 Rev. 3, Sep. 2023, doi: 10.6028/NIST.SP.800-82r3.
- [2] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, Aug. 2020, doi: 10.6028/NIST.SP.800-207.
- [3] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," IETF RFC 2104, Feb. 1997, doi: 10.17487/RFC2104.
- [4] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hammin, "Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations," NIST AI 100-2e2025, Mar. 2025, doi: 10.6028/NIST.AI.100-2e2025.
- [5] D. I. Urbina et al., "Limiting the Impact of Stealthy Attacks on Industrial Control Systems," in *Proc. ACM CCS*, 2016, pp. 1092–1105, doi: 10.1145/2976749.2978388.
- [6] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection," in *Proc. NDSS*, 2018, doi: 10.14722/ndss.2018.23211.
- [7] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020, doi: 10.1109/ACCESS.2020.3022862.
- [8] A. P. Mathur and N. O. Tippenhauer, "SWaT: A Water Treatment Testbed for Research and Training on ICS Security," in *Proc. CySWater*, 2016, pp. 31–36, doi: 10.1109/CySWater.2016.7469060.
- [9] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A Dataset to Support Research in the Design of Secure Water Treatment Systems," in *Critical Information Infrastructures Security*, LNCS 10242, 2017, pp. 88–99, doi: 10.1007/978-3-319-71368-7_8.
- [10] A. Deng and B. Hooi, "Graph Neural Network-Based Anomaly Detection in Multivariate Time Series," in *Proc. AAAI*, vol. 35, no. 5, pp. 4027–4035, 2021, doi: 10.1609/aaai.v35i5.16523.
- [11] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: UnSupervised Anomaly Detection on Multivariate Time Series," in *Proc. ACM SIGKDD*, 2020, pp. 3395–3404, doi: 10.1145/3394486.3403392.
- [12] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data," *Proc. VLDB Endow.*, vol. 15, no. 6, pp. 1201–1214, 2022, doi: 10.14778/3514061.3514067.
- [13] K. G. Anagnostakis, S. Sidiroglou, P. Akritidis, K. Xinidis, E. Markatos, and A. D. Keromytis, "Detecting Targeted Attacks Using Shadow Honey pots," in *Proc. 14th USENIX Security Symposium*, 2005, pp. 129–144.
- [14] The HoneyNet Project, "Conpot ICS/SCADA Honey pot Documentation," version 0.6.0. [Online]. Available: <https://conpot.readthedocs.io/>
- [15] M. Nawrocki, M. Wählisch, T. C. Schmidt, C. Keil, and J. Schönfelder, "Don't Get Stung, Cover Your ICS in Honey: How Do Honey pots Fit within Industrial Control System Security," *Computers & Security*, vol. 114, 2022, Art. no. 102598, doi: 10.1016/j.cose.2021.102598.
- [16] J. Williams, M. Edwards, and J. Gardiner, "Time-to-Lie: Identifying Industrial Control System Honey pots Using the Internet Control Message Protocol," arXiv:2410.17731, 2024.